Tabular RL vs DQN

Personal study guide: understand the whole task

Intelligent Agents · Project by Aispur Santiago and Amaya Bryan · Instructor: PhD Armas Rolando

Prepared 4 November 2026 from the project repository `rl_dqn_project`

How to use this guide

This is for **you**, not for submission. It explains what the assignment asks, the ideas behind it, what our code does, what we found, and how to survive the oral defense (50% of the grade). Every number comes from the repository (`ANALYSIS.md`, `results/analysis.md`) and was re-checked today by running `analysis.py` (it reproduced the same values).

Read in this order: **1 The task → 2 Ideas → 3 Experiments and results → 4 Code → 5 Defense**. Chapter 6 is a one-page cheat sheet.

Contents

1 The task in plain words . . . 3
1.1 What the professor wants . . . 3
1.2 The three rules that decide marks . . . 3
1.3 Required components and where we satisfy them . . . 3
2 The ideas you must master . . . 4
2.1 Reinforcement learning in one paragraph . . . 4
2.2 Tabular Q-learning . . . 4
2.3 DQN (Mnih et al., 2015) . . . 4
2.4 Why DQN can be more sample-efficient: generalization . . . 4
2.5 The deadly triad . . . 5
2.6 Why tabular and DQN both fail in complex environments (theory part) . . . 5
3 Experiments and results . . . 6
3.1 Tier 1: CartPole-v1 (4-D state, 2 actions, +1 per step, solved ≥ 475) . . . 6
3.2 Tier 1: MountainCar-v0 (sparse reward, planned negative result) . . . 7
3.3 Tier 2: LunarLander-v3 (8-D state, 4 actions, solved ≥ 200) . . . 9
3.4 Methodology choices (be ready to justify each) . . . 10
4 The code, file by file . . . 11
4.1 The DQN update, line by line (`DQNAgent._update`) . . . 11
4.2 Reproducibility contract . . . 11
5 Surviving the oral defense . . . 12
5.1 Likely questions and strong answers . . . 12
5.2 Live-modification recipes (each: one edit + a `--quick` run, 1–3 min) . . . 12
6 Compute and time taken . . . 14
6.1 Fresh timings measured today (4 Nov 2026) . . . 14
7 One-page cheat sheet . . . 16

1 The task in plain words

1.1 What the professor wants

The assignment is **not** mainly "write an agent that wins". It is an exercise in **experimental method and scientific defense**: build fair experiments, explain why an algorithm works or fails, and defend every line of code in person. Deliverables:

1. A **6–8 page report**, written like a conference paper (ours: `report/main.pdf`, 8 pages).
2. A **code repository** that regenerates every figure with one command (`python experiments/run_all.py`).
3. An **oral defense** (50% of the grade). Both of us must be able to explain the entire codebase, including AI-generated parts. Inability to explain it carries severe penalties. Expect **live modifications** (change the ε-decay, change bin boundaries).
4. A written **theoretical analysis** of the limits of tabular RL and DQN (in the report).

1.2 The three rules that decide marks

Defense rule	Explain every line, and be ready to modify the code live.
Metric rule	Never compare on "episodes". Use **cumulative environment steps** on the x-axis. Tabular episodes last 20 steps early on; good DQN episodes last 500, so episodes are an unfair clock.
Baseline rule	Negative results are valid. DQN failing on MountainCar earns full marks **if the failure is diagnosed**.

1.3 Required components and where we satisfy them

Requirement	What we did	Where
Tier 1: tabular vs DQN	CartPole-v1: 6/10/20 bins, angle-boundary study, DQN ε-decay 5k/25k/100k. MountainCar-v0 for the sparse-reward negative result.	`experiments/tier1_*.py`
Tier 2: ablations	LunarLander-v3: no target net, no replay buffer; plus Double and Dueling DQN.	`experiments/tier2_ablation.py`
≥3 (prefer 5) seeds, confidence bands	**10** seeds per config, 95% Student-t bands (`fill_between`).	`common/stats.py`, `plotting.py`
Evaluate greedily during training	$\varepsilon = 0$, frozen weights, every 5k (10k for LunarLander) steps, plotted separately from training returns.	`common/loop.py`
Justify velocity clipping	Measured max $\lvert\dot{x}\rvert = 2.83$, $\lvert\dot{\theta}\rvert = 2.92$ → clip at 3.0 / 3.5.	`experiments/obs_ranges.py`
Catastrophic forgetting	Metric "drop = best − final"; 20/20 fast/medium DQN seeds hit 475 then fell below 200.	`analysis.py`
Wall-clock / CPU hours	Logged per run; 170 runs = 22.3 CPU-hours.	Chapter 7 of this guide
Theory: limits of both	5^6 actions, 10^{17} table cells, 256^{7056} pixel states; PPO/SAC/TD3.	Report §Theory
Reproducible repo	Fixed seeds, one command, pinned `requirements.txt`, JSON cache.	`README.md`

2 The ideas you must master

2.1 Reinforcement learning in one paragraph

An **agent** observes a state s, picks an action a, receives a reward r and a next state s'. It wants to maximize the discounted return $\sum_t \gamma^t r_t$. The **Q-function** $Q(s,a)$ is the expected return of taking a in s and then acting well. If we know Q, the best policy is $\arg\max_a Q(s,a)$ (the **greedy** policy).

ε-greedy: with probability ε take a random action, otherwise the greedy one. ε is annealed from 1 to a small value (our schedule is linear over environment steps).

2.2 Tabular Q-learning

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\Big]$$

Part in brackets = **TD error**: (what we now think the value is) minus (what we believed). The environments have continuous states, so we must **discretize**: split each dimension into k bins and combine bin ids into one integer (our `Discretizer`, `np.searchsorted` + `ravel_multi_index`). The Q-table has `n_states × n_actions` entries.

> **Two details graders love**
> **Terminated vs truncated.** On a true terminal transition (pole fell) the target is just r (no bootstrap). On a time-limit cut (500 steps in CartPole) the pole did **not** fall, so we still bootstrap. Mixing these up silently biases values.
> **Off-policy.** The behavior policy is ε-greedy, but the target uses `max` (the greedy policy). SARSA would use the action actually taken next.

2.3 DQN (Mnih et al., 2015)

Replace the table with a neural net $Q_\theta(s,\cdot)$ that outputs one value per action. Train it by regression toward the target

$$y = r + \gamma(1-d)\max_{a'} Q_{|\theta|}(s',a'), \quad \text{loss} = \text{Huber}(Q_\theta(s,a), y)$$

Two tricks make this stable:

- **Replay buffer**: store transitions, train on random mini-batches. This breaks the correlation between consecutive samples (SGD assumes roughly i.i.d. data) and reuses each transition many times.
- **Target network** $|\theta|$: a frozen copy updated only every C steps (hard copy via `load_state_dict`), or slowly (Polyak: $|\theta| \leftarrow \tau\theta + (1-\tau)|\theta|$). Without it, the target moves with every gradient step: the network chases itself.

Extensions. Double DQN: the online net **selects** the next action, the target net **evaluates** it (reduces the overestimation bias of `max`). Dueling DQN: split into $V(s)$ and advantage $A(s,a)$, $Q = V + A - \text{mean}(A)$.

2.4 Why DQN can be more sample-efficient: generalization

A table cell is updated only when **that exact cell** is visited. A network shares weights, so one gradient step also changes Q for **nearby** states. That is the entire source of DQN's sample-efficiency win, and (see 2.5) also the source of its instabilities.

2.5 The deadly triad

Function approximation + bootstrapping + off-policy data can make values **diverge** (Sutton & Barto §11.3). We saw it in MountainCar: true values are all ≤ 0, yet some DQN runs reached Q = +95,000. A table cannot do this because each update moves only its own cell.

2.6 Why tabular and DQN both fail in complex environments (theory part)

Case	Number	Meaning
HalfCheetah, 6-D action, 5 levels per joint	$5^6 = 15{,}625$ actions	DQN's output layer needs 15,625 nodes and a max over all of them; ε-greedy over 15k arms is hopeless.
Ant, 8-D action, 5 levels	$5^8 \approx 390{,}625$	Worse.
17-D state, 10 bins per dimension	10^{17} cells	800 PB per action in float64; most cells never visited.
Atari 84×84 grayscale	$256^{7056} \approx 10^{16992}$ states	Exact revisits never happen; a table can never reuse an update.

What to use instead. Continuous control: actor–critic methods that output actions directly: **PPO** (on-policy, Gaussian policy, clipped updates), **SAC** (off-policy, entropy-regularized), **TD3** (deterministic actor, twin critics to curb overestimation). Pixels: convolutional networks that generalize across similar frames.

3 Experiments and results

All numbers: mean ± 95% t-CI over **10 seeds**; "greedy" = ε = 0, frozen weights. Tabular got 1M steps on CartPole, DQN 200k; MountainCar and LunarLander 500k each.

3.1 Tier 1: CartPole-v1 (4-D state, 2 actions, +1 per step, solved ≥ 475)

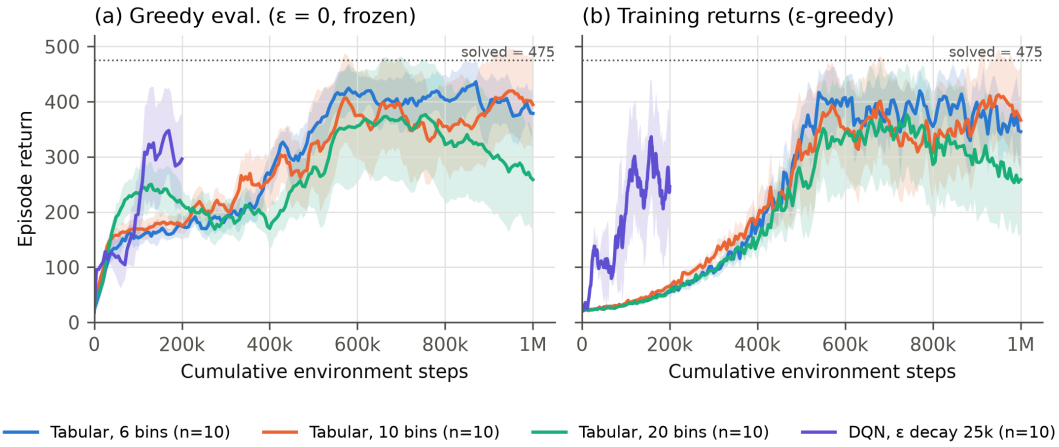


Figure 1: Greedy evaluation (left) and training returns (right) on a common step axis. DQN rises within 100k steps; tables need 400–550k.

Config	Final	AUC	Steps to 475	Solved
Tabular 6 bins	356 ± 51	303 ± 11	518 ± 59k	10/10
Tabular 10 bins	**392 ± 81**	294 ± 30	382 ± 58k	10/10
Tabular 20 bins	263 ± 96	269 ± 47	505 ± 70k	7/10
Angle: 2 bins	305 ± 73	226 ± 23	608 ± 159k	8/10
Angle: fine near 0	362 ± 74	274 ± 31	474 ± 114k	6/10
DQN ε 5k	340 ± 134	**324 ± 36**	**56 ± 16k**	10/10
DQN ε 25k	297 ± 115	223 ± 34	98 ± 21k	10/10
DQN ε 100k	140 ± 66	103 ± 9	180k	2/10

Key findings

1. **Sample efficiency vs compute.** DQN needs 56k steps to reach 475 vs 382k for the best table (**6.9×**). But a DQN step costs 2.24 ms CPU vs 0.097 ms (**23×**), so reaching 475 costs 124 CPU-s for DQN vs 37 s for the table (the table is **3.4× cheaper in compute**). "Which is faster" depends on whether samples or computation are scarce. At an equal 200k budget DQN scores 356 ± 95 vs 175–215 (Welch p = 0.002).
2. **Number of bins = bias–variance.** 6 bins (1,296 cells) is too coarse to express a precise policy; 20 bins (160,000 cells) is data-starved (only 4.6% of cells ever visited). 10 is the compromise. The 6-vs-20 final-score difference is only a trend (p = 0.074).
3. **Evaluate during training.** All 20/20 fast- and medium-decay DQN seeds reached ≥ 475 and later collapsed below 200 (**catastrophic forgetting**). Judged only at the end, a solved agent looks mediocre. Cause: once the policy is good, the buffer holds only near-upright states and the network overwrites what it knew about large tilts.
4. **ε-decay.** Fast decay is best here, because random actions already cover the relevant states. A 100k decay spends half the run nearly random: only 2/10 seeds solve it.

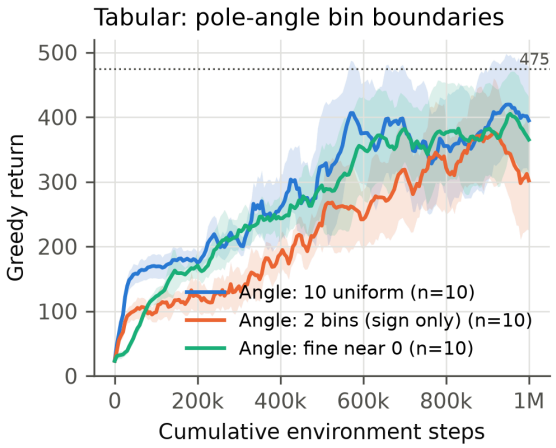


Figure 2: Pole-angle boundary study (other dimensions fixed at 10 bins). Sign-only angle is clearly slower early ($p < 10^{-6}$).

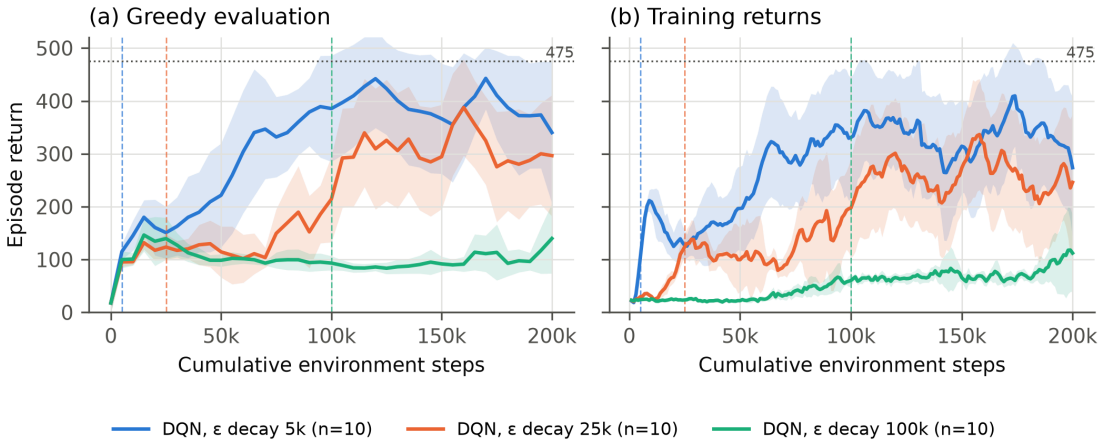


Figure 3: DQN ε-decay study: fast decay wins on CartPole; drops after the peak are forgetting.

Velocity clipping is justified by data. Cart velocity and pole angular velocity are unbounded in Gymnasium. Measured over 50k steps under a random and a trained policy, the maxima are 2.83 and 2.92, inside our bounds 3.0 and 3.5. So no visited value gets clipped and no bin is wasted.

3.2 Tier 1: MountainCar-v0 (sparse reward, planned negative result)

Reward is −1 every step until the flag; the episode is truncated at 200 steps. A random policy almost never reaches the flag.

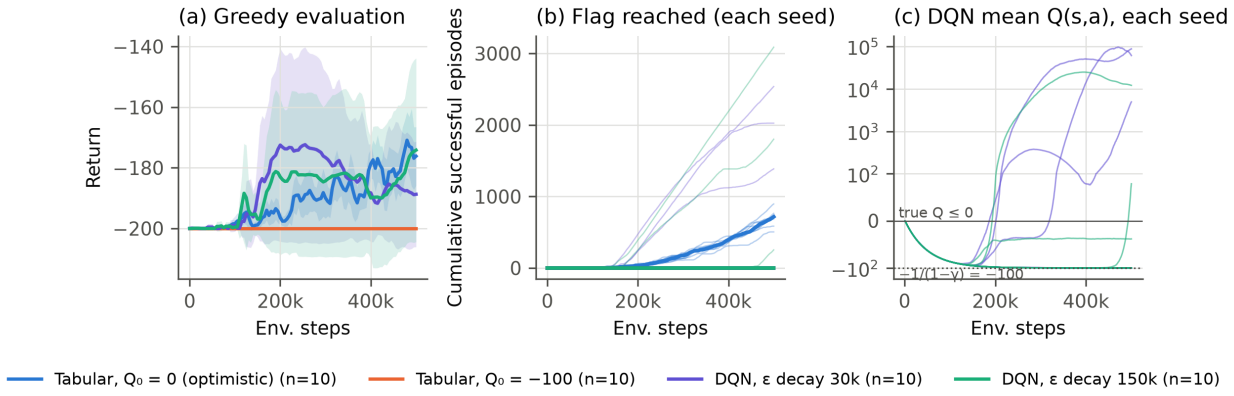


Figure 4: MountainCar: (a) greedy return, (b) seeds that found the flag, (c) mean Q. Before the first flag every DQN sits at Q = −100.

Config	Final greedy	Best	Found flag	Mean Q at end
Tabular $Q_0 = 0$ (optimistic)	−176 ± 10	−148 ± 6	**10/10**	—
Tabular $Q_0 = -100$	−200 ± 0	−200 ± 0	**0/10**	—
DQN ε 30k	−189 ± 17	−168 ± 34	3/10	diverged in 3/3
DQN ε 150k	−174 ± 30	−164 ± 32	3/10	mixed

Why DQN fails (14 of 20 runs). Until the flag is seen, every target is $-1 + \gamma \max Q$, whose fixed point is $-\frac{1}{1-\gamma} = -100$ for **every** state: “all states are equally bad”. The network sees no gradient toward the goal. Those 14 runs end with mean Q = −99.4 ± 0.1 and greedy return exactly −200.

Why the optimistic table succeeds. All true values are negative, so $Q_0 = 0$ is an over-estimate. Each visit lowers the tried action’s value, so untried actions look best: **directed exploration**. With $Q_0 = -100$ the identical algorithm finds the flag 0/10 times. The cause is the initialization, not “tabular vs deep”.

When DQN does find the flag (6/20), Q diverges (up to +95,715 although true Q ≤ 0, gradient norm from 0.4 up to 7,861). The flag creates a sharp value cliff that a smooth MLP cannot fit locally; error leaks to neighbours and `max` amplifies it (deadly triad). A unit test checks that terminal targets are $y = r$, so it is not a done-flag bug. The greedy policy still improves (−138 ± 38, better than the table) because argmax only depends on the **ranking** of Q.

Takeaway. Sparse rewards need directed exploration (optimism, count bonuses, intrinsic motivation); DQN there also needs divergence safeguards (normalized observations, lower learning rate, Double DQN, soft target updates).

3.3 Tier 2: LunarLander-v3 (8-D state, 4 actions, solved ≥ 200)

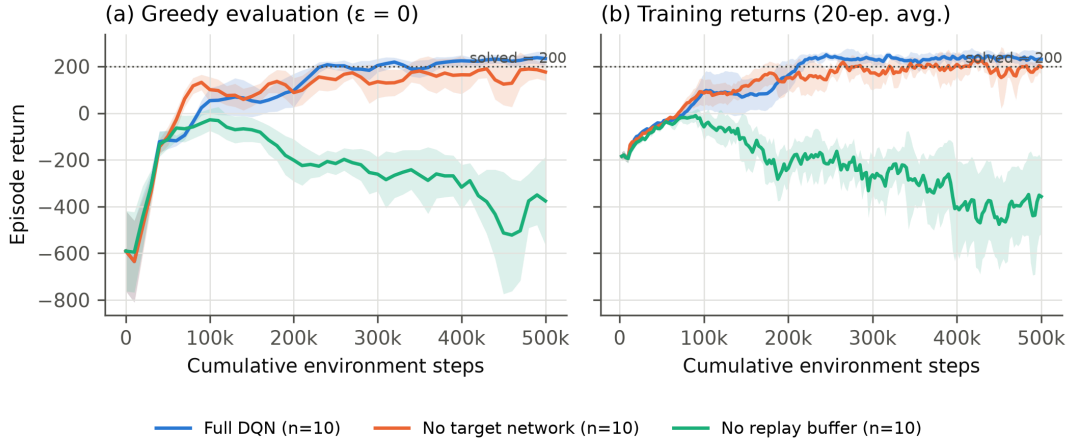


Figure 5: Ablations: full DQN vs no target network vs no replay buffer.

Config	Final	Best	Drop	p vs full	Mean Q	CPU min
Full DQN	234 ± 29	273	39	—	43.4	10.4
No target net	176 ± 37	275	99	0.014	48.4	10.0
No replay	−376 ± 188	149	525	4e−5	256.9	24.0
Double DQN	234 ± 29	276	42	0.98	43.2	10.2
Dueling DQN	238 ± 33	276	38	0.82	46.9	9.8

- **Target network = stability, not speed.** Without it learning is as fast and peaks equally high (275 vs 273), but 10/10 seeds that reached ≥ 200 later fell below 0 (2/10 with a target net). The TD loss stays higher: the net chases its own moving target.
- **Replay buffer = learning at all.** Online batch-1 updates peak at 149 and decay to −376. Mean Q is 5.9× larger than the full agent's while the policy is far worse (overestimation), gradients reach 300 (vs 20). Reasons: correlated data breaks i.i.d. SGD; each transition is used once; each update overwrites knowledge of other regions (catastrophic interference). It is also the most expensive (24 vs 10 CPU-min) because it updates on every step instead of every 4th.
- **Double / Dueling: null result, reported honestly.** No measurable difference (p = 0.98, 0.82). With 4 actions, a 1,000-step target refresh and Huber loss, the overestimation Double DQN fixes is already small. The original gains were on Atari.

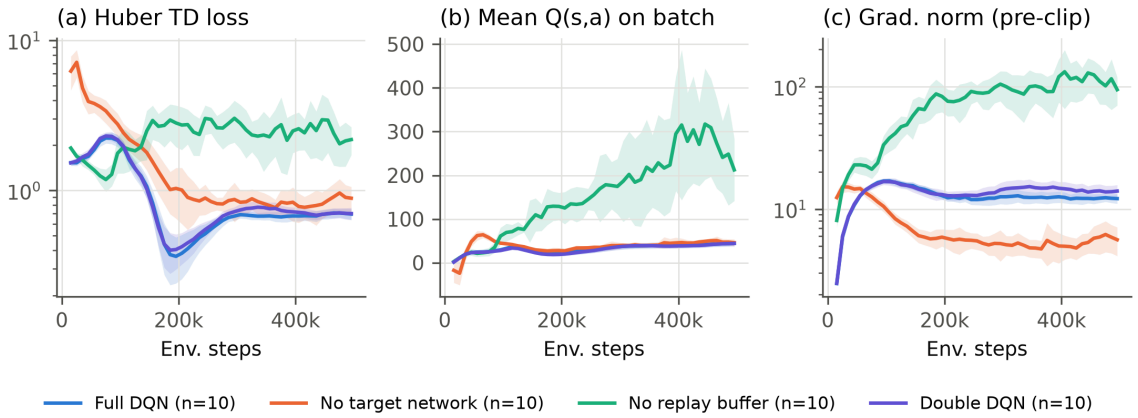


Figure 6: Diagnostics: TD loss, mean Q and gradient norm per variant.

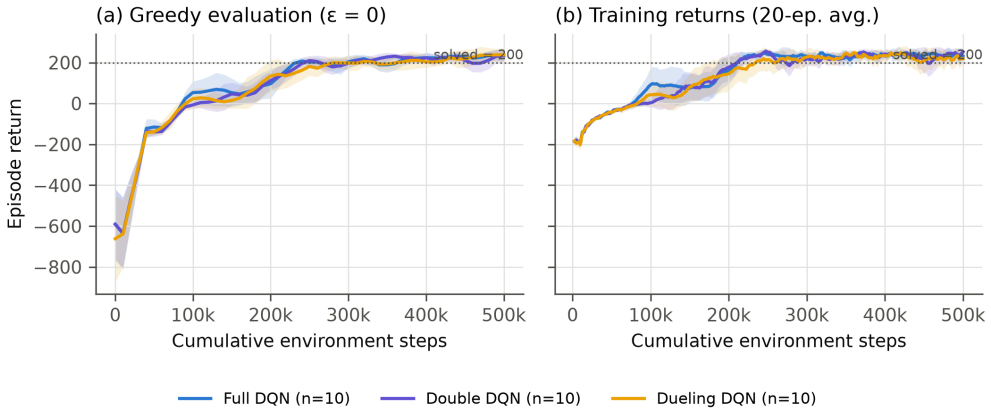


Figure 7: Double and Dueling DQN vs full DQN: overlapping curves.

3.4 Methodology choices (be ready to justify each)

Choice	Why
x-axis = environment steps	Fair clock for agents that consume data at different episode rates.
Greedy eval every N steps, fixed eval seeds	Separates what the policy knows from exploration noise; same start states at every checkpoint, so curves reflect the policy, not luck.
10 seeds, Student-t CI ($t_9 = 2.26$)	Honest bands for small n (wider than 1.96); Welch t-tests for key claims.
Huber loss + grad-norm clip 10	Bounded gradients for large early TD errors (Mnih et al.).
CPU time from one machine only	Seeds 5–9 ran on GitHub Codespaces; timings across CPUs are not mixed (`results/hosts.json`).
Report p-values, say “trend” if p > 0.05	Going from 5 to 10 seeds reversed our 6-vs-10-bins ranking: rigor changes conclusions.

Threats to validity (a reviewer could raise these)
No per-method hyper-parameter tuning (we claim **mechanisms**, not exact scores). Budgets differ by design on CartPole (tables 1M, DQN 200k); equal-budget comparison is made at 200k. Two machine types. Display smoothing in plots (tables use raw values). Several Welch tests without multiple-comparison correction (headline p-values survive; p = 0.014 and 0.074 are moderate evidence and a trend).

4 The code, file by file

`agents/tabular.py`	`TabularAgent`: ε-greedy, random tie-breaking among equal Q (otherwise action 0 is always chosen at a zero table), the Q-learning update, optimistic `init_value`.
`agents/dqn.py`	`QNetwork` (MLP, optional dueling head), `ReplayBuffer` (circular preallocated arrays, stores only `terminated`), `DQNAgent` (switches: target network, replay, double).
`envs/discretizers.py`	Bins → one integer state id. Hand-made edges via `edges={dim: [...]}`.
`common/loop.py`	One training loop for both agents (fair by construction); global step counter `t` drives ε, learning-start, target-update and evaluation.
`common/runner.py`	Parallel (config × seed) runs, JSON cache with atomic writes, saves weights.
`common/stats.py`	Step-aligned curves, t-CI, summaries (final, best, AUC, drop, steps to threshold).
`experiments/*.py`	One script per study; flags `--quick`, `--plot-only`, `--force`, `--seeds`, `--seed-start`, `--workers`.
`analysis.py`	Every statistic in the report (Welch tests, efficiency ratios, divergence).
`tests/test_core.py`	10 sanity tests: update rules, gather, targets, terminal handling, bit-for-bit reproducibility.

4.1 The DQN update, line by line (`DQNAgent._update`)

```
q_all = self.q(s)                                  # [B, n_actions]
q_sa  = q_all.gather(1, a.unsqueeze(1)).squeeze(1) # Q of the action actually TAKEN
with torch.no_grad():                              # target is a constant (semi-gradient)
    target_net = self.q_target if self.use_target else self.q   # ablation switch
    # Double DQN: online net selects, target net evaluates
    q_next = target_net(s2).max(1).values
    y = r + gamma * (1 - done) * q_next            # done = terminated only
loss = F.smooth_l1_loss(q_sa, y)                   # Huber
clip_grad_norm_(self.q.parameters(), 10)           # second safety net
```

(Condensed from `_update` and `td_targets` in `agents/dqn.py`; the real code splits the target into `td_targets`.) `gather` picks, for each row i, column $a[i]$, so we regress the Q-values of the **taken** actions. The `max` is used only to build next-state targets.

4.2 Reproducibility contract

For seed k: `random`, `numpy`, `torch.manual_seed(k)`, the agent's own `default_rng(k)`, `env.reset(seed=k)` and `env.action_space.seed(k)`. Evaluation episodes use disjoint seeds `1_000_000 + 1000k + i`, identical at every checkpoint. Each result JSON stores the full spec and library versions, and a test checks that two runs with the same seed match bit for bit.

5 Surviving the oral defense

> **Mindset**
> You need to explain **why**, not only **what**. For each result, be able to say: the number, the mechanism, and the figure that shows it.

5.1 Likely questions and strong answers

“Change the pole-angle bin boundaries. What happens?” Too coarse around vertical: cannot tell a slight tilt from upright, reacts late, falls (our 2-bin sign-only run). Too fine: more cells to fill, so slower early (each extra bin multiplies the table). Our `tabular_angle_fine_center` puts resolution near 0 where the pole spends its time.

“Explain `gather()`.” Picks $Q(s_i, a_i)$ for the actions taken, not the max. The max is only for the next-state target.

“How is the target network updated?” Hard copy with `load_state_dict` every 500 (CartPole) / 1000 (LunarLander) environment steps; `tau < 1` switches to Polyak averaging.

“Why does no-target oscillate?” Each gradient step also moves its own target; similar states share weights, so raising Q(s,a) raises Q(s’,·): a feedback loop visible as higher TD loss and gradient norms.

“Why does no-replay fail?” Consecutive transitions are nearly identical; SGD sees a correlated, non-stationary stream, overfits the current part of the trajectory and forgets the rest; each transition is used once.

“Why steps and not episodes?” Early tabular episodes last 15–20 steps, good DQN episodes 500.

“DQN learns in fewer steps; why is it slower on the clock?” A forward pass to act plus a backward pass on a batch of 64–128 each step, vs one table lookup. 23× more CPU per step.

“Q = +90,000 in a task with rewards of −1. Bug?” No. Terminal targets are tested ($y = r$); divergence starts only after the first flag. Deadly triad. Fixes we would try: normalize observations, lower learning rate, Double DQN, soft target ($\tau = 0.005$).

“Double DQN did nothing. Wrong implementation?” It selects with the online net and evaluates with the target net (tested). Overestimation is small here (mean Q 43.4 vs 43.2); Atari has 18 actions and noisier estimates.

“Is it statistically meaningful?” 10 seeds, t-CIs, Welch tests; where intervals overlap we say trend or null.

“What for HalfCheetah, BipedalWalker or Atari?” PPO, SAC or TD3 for continuous actions; a CNN for pixels (numbers in 2.6).

5.2 Live-modification recipes (each: one edit + a `--quick` run, 1–3 min)

Request	Edit	Run
Change ε-decay	`experiments/tier1_cartpole.py`: `eps_decay_steps=...`	`python experiments/tier1_cartpole.py --quick`
Change number of bins	`tab(14)` in the config dict	same
Angle boundaries	`angle_edges=[...]` (sorted, radians)	same
Velocity clip range	`envs/discretizers.py`: `CARTPOLE_LOW/HIGH`	same

Soft target update	`tau=0.005`	`python experiments/tier2_ablation.py --quick`
Target update frequency	`target_update_freq`	same
MSE instead of Huber	`agents/dqn.py`: `F.mse_loss(q_sa, y)`	any `--quick`
Different γ	`gamma: 0.9`	any `--quick`

Revert afterwards with `git checkout -- <file>`.

6 Compute and time taken

Whole project: 170 runs, 22.3 CPU-hours (23.1 wall-hours summed over runs), one thread per run. Seeds were split across the laptop (Ryzen 5 7235HS) and two GitHub Codespaces to meet the deadline; with 8 threads the full experiment suite is 3 hours.

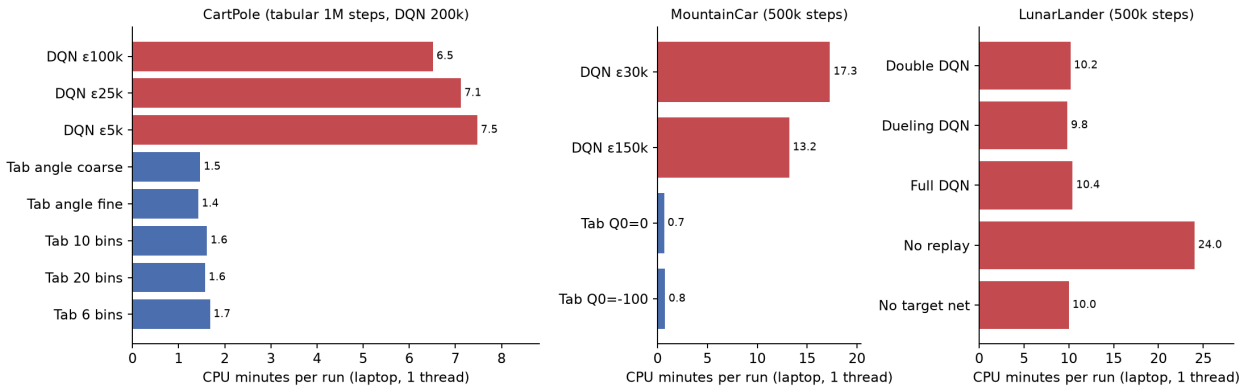


Figure 8: CPU minutes per run on the reference laptop (seeds that ran on the laptop only). Blue = tabular, red = DQN.

Reading the chart. A 1M-step table costs 1.5 CPU-min; a 200k-step CartPole DQN 7 min (about 5× more for one-fifth the steps). MountainCar DQN takes 13–17 min vs 0.7 min for the table. In LunarLander, no-replay costs 24 min vs 10 for everything else because it updates every step.

6.1 Fresh timings measured today (4 Nov 2026)

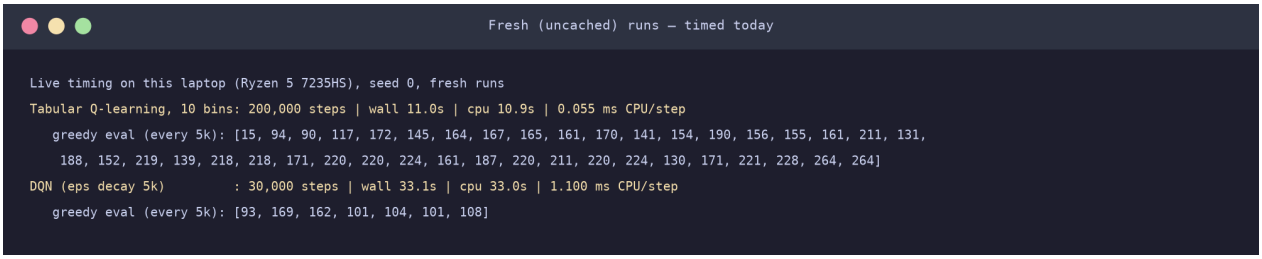


Figure 9: New, uncached runs of seed 0 on this laptop. Tabular 10 bins: 200k steps in 11 s. DQN: 30k steps in 33 s (1.1 ms/step). This short run is not directly comparable with the archived 200k-step average of 2.24 ms/step, which is the reference figure used in the report.

Command run today	Wall time
`python -m pytest -q tests` (10 tests pass)	5.2 s
`python analysis.py`	1.3 s
`tier1_cartpole.py --plot-only`	2.6 s
`tier1_mountaincar.py --plot-only`	1.9 s
`tier2_ablation.py --plot-only`	2.7 s

The table is the practical point: all heavy work is cached in JSON, so the professor can regenerate every figure in seconds, or the whole study in 3 h with `run_all.py`.

7 One-page cheat sheet

The 12 numbers
CartPole: DQN 56k vs tabular 382k steps to 475 (6.9×). CPU/step 2.24 ms vs 0.097 ms (23×). CPU-s to 475: 124 vs 37 (tabular 3.4× cheaper). Bins visited after 1M steps: 38.7% / 18.0% / 4.6% (6/10/20 bins).
MountainCar: optimistic table 10/10 flags, pessimistic 0/10, DQN 6/20 (14 never). Flat Q = −99.4 ≈ −100. Divergence up to +95,715.
LunarLander: no target 176 vs 234 ($p = 0.014$), 10/10 collapse; no replay −376 ($p = 4e{-}5$), Q 5.9× inflated; Double/Dueling $p = 0.98/0.82$.
Totals: 170 runs, 22.3 CPU-hours, 10 seeds, $t_9 = 2.26$.

The 8 sentences that explain everything
1. DQN wins on samples because it generalizes; tabular wins on compute because a step is cheap.
2. Bins trade precision against data: 10 beat 6 and 20.
3. Evaluate during training, because DQN forgets after it solves the task.
4. Optimism is exploration: $Q_0 = 0$ vs −100 flips 10/10 to 0/10.
5. Sparse rewards make DQN's Q converge to −100 everywhere, so there is no gradient to follow.
6. Function approximation + bootstrapping + off-policy data can diverge.
7. The target network buys stability; the replay buffer buys learning itself.
8. Report null results (Double, Dueling) and say "trend" when $p > 0.05$.

Reproduce: `python experiments/run_all.py` (3 h on 8 threads, cached runs skipped) · `python analysis.py` · `python -m pytest -q tests` · `cd report && latexmk -pdf main.tex`.